

# Lexical Scope

Imagine a **company office building**.

Each employee:

- Has access to **their desk**
- Can access **department resources**
- Can access **company-wide resources**

But:

- HR cannot access Engineering's private files.
- Intern cannot access CEO's vault.

Access is determined **by where you are located physically**.

**Scope is decided by WHERE code is WRITTEN, not where it is called.**

This is called **Lexical Scope**.

Lexical scope means that a function's scope is determined by where it is defined in the source code, not where it is called. Functions can access variables from their own scope and their outer (parent) scopes.

*Functions can access variables from their own scope and from the scope in which they were defined.*

```
function outer() {  
  let name = "Prajwal";
```

```
  function inner() {  
    console.log(name); // Accessible because of lexical scope  
  }
```

```
  inner();  
}
```

```
outer();
```

✅ inner() can access name because it was **defined inside outer()**.

👉 Lexical scope **does NOT** mean “look upward.”

👉 It means “**who your parent is was decided when you were created.**”

JavaScript permanently records:

inner's lexical parent = outer's environment

JavaScript permanently records:

inner's lexical parent = outer's environment

This never changes. Ever.

So later when inner() runs, it already knows:

"I belong to outer → I can use its variables."

**Q1: Does JavaScript use lexical or dynamic scope?**

**Answer:** Lexical. Determined at definition time.

**Q2: What creates scope?**

- Functions
- Blocks (let, const)
- Modules

```
let a = 10;
```

```
function test() {  
  console.log(a);  
}
```

```
function run() {  
  let a = 20;  
  test();  
}
```

```
run();
```

here even test () is called inside run where a=20 in parent scope still it prints a=10 because of lexical scope it is written outside run and a=10 in its parent env.

#### **4. Explain the Scope Chain in Lexical Scope**

When a variable is used:

JS searches in this order:

Current Scope → Parent Scope → Grandparent → Global → Error